

PART 1: Conceptual

Explain informally but clearly and precisely.

- (a) Closed world assumption.

Solution: The assumption that when a proposition cannot be proven true, it is taken as false (like in a relational database: if the tuple is not in the table, it is not true).

- (b) The frame problem. Provide two examples.

Solution:

The challenge of representing the effects of action in logic without having to represent explicitly a large number of intuitively obvious non-effects. See [this entry](#) in Stanford Encyclopedia of Philosophy.

- (c) The three components an action using the STRIPS language? Explain, in your own words, each of the components are meant to define and capture. Give an example of a STRIPS action.

Solution: A STRIPS action is built from three lists: precondition, add, and delete lists.

Example [here](#)

- (d) At least 2 limitations of STRIPS language to represent actions.

Solution: Cannot represent stochastic actions, or actions that have conditional effect.

- (e) The differences and similarities between problem solving/search and planning.

Solution:

Both are concerned with getting from a start state to a goal using a set of defined operations or actions.

In planning, however, we use a structured language to represent states, goals, and plans, which allows for generic solver algorithms that among other things can automatically extract heuristic functions. In search, we need to hard-code the heuristic function depending on which particular problem we are solving.

- (f) What is the difference between path/motion planning and automated planning.

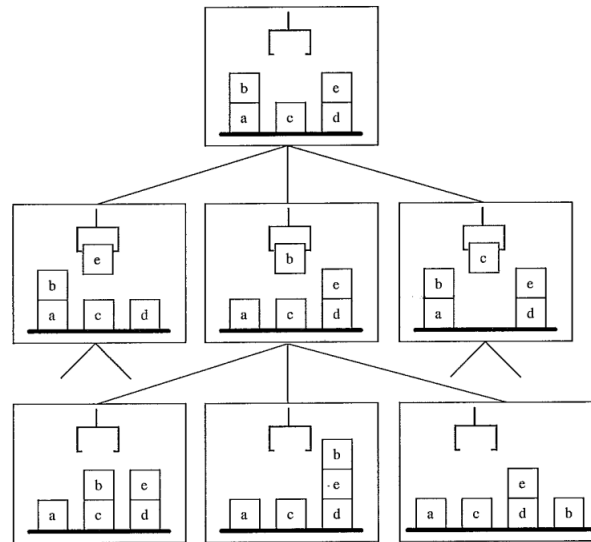
Solution: Path planning is a domain dependent task: it is known what the problem is, namely, navigating from source to destination in a network (of roads, for example). Automated planning implies not knowing what the problem is, it could be a motion planning task or playing tic-tac-toe, or a robot collecting and delivering mail in a building.

- (g) Explain what GraphPlan is and what it is used for.

Solution: Graphplan is a technique to do planning by constructing a layered graph and is also used to obtain, automatically, domain-independent heuristics from a domain problem specification given in a certain factored representation (like STRIPS or PDDL).

PART 2: Blocks World

Consider the following blocks world scenario:



Suppose we use the following fluents (predicates in logic) to represent a states:

- $OnTable(x)$: block x is on the table.
- $On(x, y)$: block x is on top of block y .
- $Clear(x)$: there is nothing on top of block x .
- $Gripping(x)$: the gripper is holding block x .
- $Empty$: the gripper is not holding anything.

- (a) Specify the initial state in logical representation, that is, using conjunction of atoms and relying on the close world assumption.

Solution: (Note: this is a partial solution)

The initial state of the blocks world above may be represented by the following set of predicates:

$$OnTable(a) \wedge OnTable(c) \wedge \dots \wedge On(b, a) \wedge On(e, d) \wedge Clear(b) \wedge Clear(c) \wedge Clear(e) \wedge \dots$$

- (b) Write the STRIPS representation for operators $pickup(x)$, $putdown(x)$, $stack(x, y)$, and $unstack(x, y)$. Should a block be made clear stacking or putting it down?

Solution: (Note: this is a partial solution)

$pickup(x)$:

Precondition: $Empty, Clear(x), OnTable(x)$

Add List: $Gripping(x)$

Delete List: $OnTable(x), Clear(x), Empty$

```

 $\underline{putdown(x):}$ 
  Precondition:  $Gripping(x)$ 
  Add List:     $OnTable(x), Empty, Clear(x)$ 
  Delete List:  $Gripping(x)$ 
 $\underline{stack(x, y):}$ 
  Precondition:  $Clear(y), Gripping(x)$ 
  Add List:    ....
  Delete List:  $Clear(y), Gripping(x)$ 
 $\underline{unstack(x, y):}$ 
  Precondition:  $Clear(x), Empty, On(x, y)$ 
  Add List:    ....
  Delete List: ....

```

$Clear(x)$ implies that the block is clear to be stacked on; and one cannot stack something on top of something that is in the gripper. More importantly, unless **equality constraints** are used explicitly in the PDDL description, it could lead to an action that stack a block onto itself! Why? Consider a state such that $OnTable(a) \wedge Clear(a) \wedge Gripping(b) \wedge Clear(b)$.

- (c) What is the new state if we apply operator-action $unstack(e, d)$ to the initial state.

Solution:

$OnTable(a) \wedge OnTable(c) \wedge OnTable(d) \wedge On(b, a) \wedge \mathbf{Gripping(e)} \wedge Clear(b) \wedge Clear(c) \wedge \mathbf{Clear(d)}$

- (d) Explain in English what frame axioms would be required in this domain. Explain how those frame axioms were respected in the question before when $unstack(e, d)$ was executed.

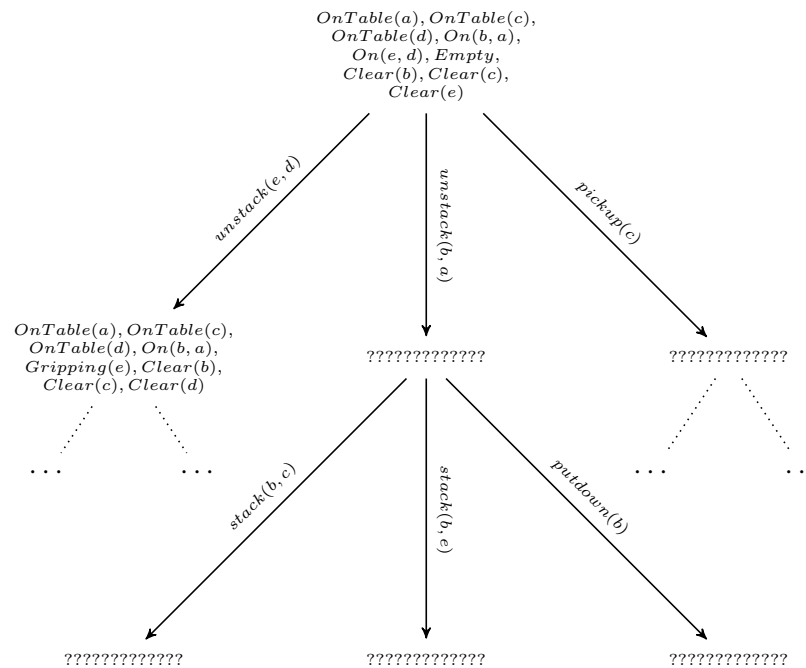
Solution: In the above, we can see as a result of applying the new operator, new predicates have been added to the new state, whereas other predicates such as $On(e, d) \wedge Empty$ have been deleted from the new state because they are no longer valid.

You will notice that predicates such as $OnTable(a) \wedge OnTable(c) \wedge OnTable(d)$ continue to remain true in the new state, thus respecting the frame axioms one would have to model the domain.

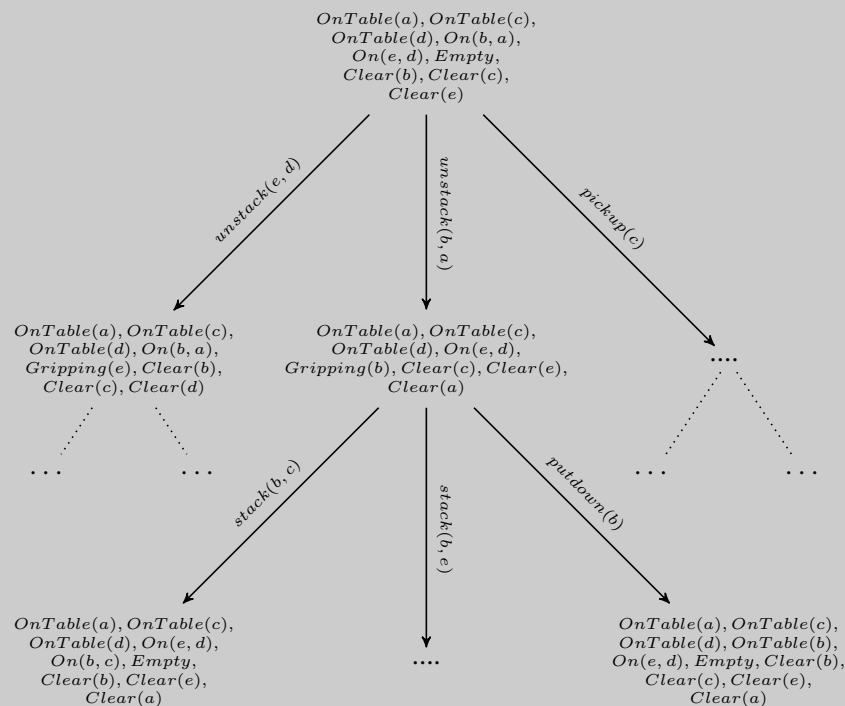
Frame axioms encode the knowledge that the things that remain the same when not affected by an action. While this comes as common sense knowledge for humans, a formal model for automatic reasoning (as computers need) require that this knowledge is encoded.

The semantics of planning languages like STRIPS and PDDL already have built-in the frame axioms: if a predicate is not affected by an operator, then its truth value remains the same.

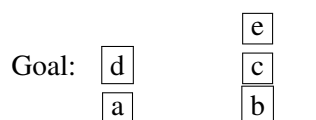
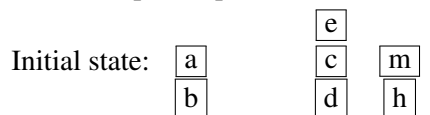
- (e) Use the operators (and the implicit frame axioms) of the previous question to complete (by filling ?????) part of the search state space of the blocks world given in the figure above. The root of the tree should be the logical initial state representation and edges should represent application of ground operators.



Solution: (Note: this is a partial solution)



(f) Show an optimal plan that solves this problem:



Solution:

There are multiple optimal solutions, here is one:

unstack(a, b), putdown(a), unstack(e, c), putdown(e), unstack(c, d), stack(c, b), pickup(e), stack(e, c), pickup(d), stack(d, a)

If you have not come up with one yourself and just read ours, now get one by yourself.

- (g) How many solutions are there for the above question? How many optimal solutions?

Solution:

There are infinitely many solutions, as you can start by stacking and unstacking the same block repeatedly before proceeding. There are only 8 optimal solutions (the first and last pairs of actions can be swapped independently, and for each of those 4 solutions, we can stack and unstack *e* with *m* instead of putting it on the table temporarily).

- (h) Is the goal state correctly defined? Explain either way.

Solution: Yes, a goal is not a full state, it is a condition that any state should satisfy. Hence, in this condition it does not matter where blocks *m* and *h* end up being.

Said so, one could argue whether the goal picture specify whether blocks *d* and *e* should be clear or not. If block *m* ends up on top of block *e*, is it a goal state?

- (i) What would GraphPlan give as heuristic value for the initial state? To find out, build the planning graph starting from the initial state. Is the value of the heuristic less, equal, or greater than the true length of the plan you found in the previous question? Explain why.

Solution:

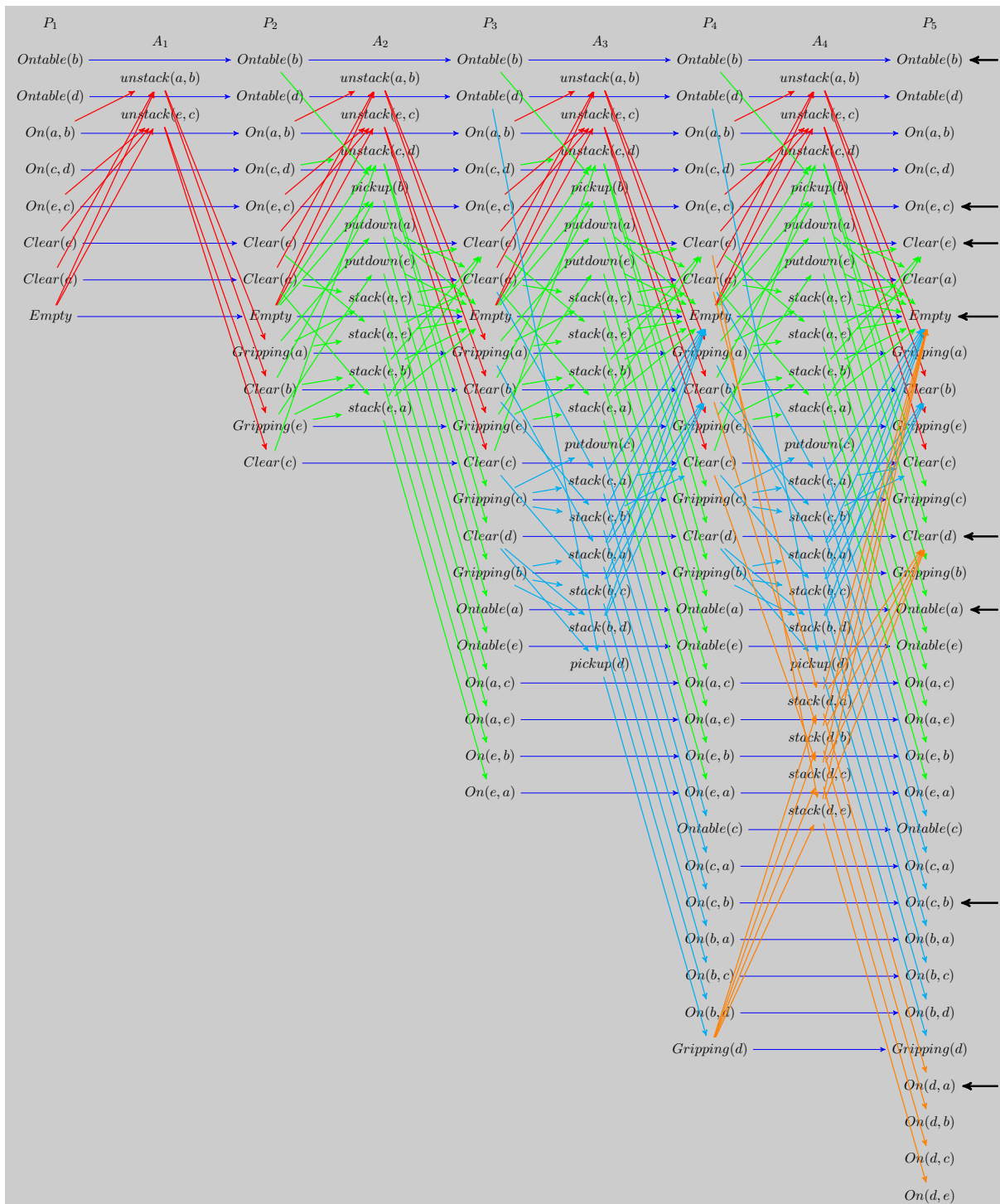
Please check this [video](#) that explains this solution in more detail.

The optimal plan has 10 actions, but GraphPlan should give you a lower heuristic value because, among other things, it will be able to hold many blocks at the same time, put them on the table all at the same time, and the original sub-tower $On(e, c)$ will always stay true!

We construct a GraphPlan graph by iteratively alternating proposition layers (P_i) and action layers (A_i). Starting with the set of initial propositions in P_1 , the graph can be constructed by the following rules:

- an action is in A_i iff all of its preconditions are in P_i ; and,
- a proposition is in P_{i+1} iff it is in the add list for an action in A_i (except for P_1).

Here is the graph representing the GraphPlan algorithm process for the this blocksworld problem (to save space we will ignore blocks *m* and *h*, as they are not useful in this instance):



Now, as the goal is $Ontable(a) \wedge Ontable(b) \wedge On(d,a) \wedge On(c,b) \wedge On(e,c) \wedge Clear(d) \wedge Clear(e)$, and all of those propositions are in layer P_5 (indicated with black arrows), we can say the heuristic lower bound for the plan is 4.

Some actions, such as those that undo previous actions (e.g., $unstack(x,y) \rightarrow stack(y,x)$) have been omitted to save space. You may have noticed that the number of propositions and actions monotonically increases with each layer. For very large planning problems, this can quickly blow up, beyond control (especially when doing it by hand!). Therefore, we can add an additional set of exclusion relations that are maintained throughout the process, which help to eliminate incompatible propositions and actions, hence reducing the graph size — but this is beyond the scope of this course.

- (j) Suppose that each block has a new property of colour. All the rest of the domain remains the same. What would you change to your representation and corresponding STRIPS modeling?

Solution: Nothing. The new property of colour is not required for any preconditions of the operators. So it is irrelevant for the problem at hand and does not need to be taken into account.

- (k) Suppose you want to represent the action of dropping an object. Its effect is that the object is on the floor and depending on whether it is fragile or not, it may or may not be broken. Can you represent such action in STRIPS representation? Either provide the representation or explain why not. Can you think of a way that you could work around this limitation by defining multiple actions?

Solution: It is not possible to represent such operator in STRIPS because it requires the ability to represent so-called *conditional effects*, that is, effects that depend on the context of where the action is applied (in our case, whether the object is fragile or not, which is represented by predicate *Fragile(x)*)

It is however possible to represent it in PDDL as PDDL supports conditional effects using the construct `(when (condition effect))` as follows:

```
(:action drop
  :parameters (?x)
  :precondition (and (Holding ?x))
  :effect (and (OnFloor(?x) (when (Fragile ?x) (Broken ?x)) )
```

If one wants to stick to STRIPS representations, the workaround is to use two actions *dropFragile(x)* and *dropNotFragile(x)*, then we could have preconditions on the fragility (or lack thereof) of the object *x* being dropped, so the effect is no longer conditional.

PART 3: Time to code in PDDL!.....

In this question, you will code a problem in PDDL and let the automated planning system find the solution for you! To do so, you will use the [planning.domain](#)'s editor, where you can code your domain and a specific problem and use an online solver to get the plan (if any exists!).

Check the video that I did [here](#) to show you how to use [planning.domain](#)'s editor and its animation facility!

For PDDL, besides the book, you may want to check the PDDL quick tutorial notes [here](#) and [here](#). Another useful material is this [PDDL by Example](#) pack of slides. Finally a nice web-based PDDL reference can be found [here](#).

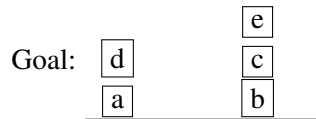
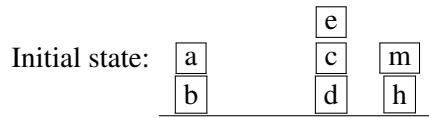
- (a) First model the Blocks World domain we have seen above. Take the template file [domain.pddl](#) encoding the domain in PDDL and fill the gaps in the editor. You can just drag and drop the file and it will load it. Remember that the domain file only contains the fluents and operators. It is meant to be usable with any specific problem instance.

Solution: Here is the start of the domain file:

```
(define (domain BLOCKS-AI20)
  (:requirements :strips)
  (:predicates (on ?x ?y)
    (ontable ?x)
    (clear ?x)
    (empty)
    (gripping ?x)
  )
```

There is now very little to be filled...

- (b) Next, encode the *specific problem* we saw above by completing the following template [problem.pddl](#) file. A problem file specifies what domain the problem is based on, the actual objects in the problem (in our case, what blocks there are), the initial state, and the condition for a state to be considered a goal state.



Solution: Your turn!

- (c) Find a solution by hitting the button **Solve!** in the web interface. If your domain and problem files are OK, you should get the plan with 10 actions!

Solution:

```
(unstack a b)
(putdown a)
(unstack e c)
(putdown e)
(unstack c d)
(stack c b)
(pickup e)
(stack e c)
(pickup d)
(stack d a)
```

- (d) Finally, animate it using the *Planimation* plugin using the [blockworld.AP.pddl](#) animation file.

Solution: Your turn!

- (e) Are you still curious? Do you want to see more involved domains with larger plans? Check the **par-cprinter** domain, from a real-world case:

This domain models the operation of the multi-engine printer, for which one prototype is developed at the Palo Alto Research Center (PARC). This type of printer can handle multiple print jobs simultaneously. Multiple sheets, belonging to the same job or different jobs, can be printed simultaneously using multiple Image Marking Engines (IME). Each IME can either be color, which can print both color and black&white images, or mono, which can only print black&white image. Each sheet needs to go through multiple printer components such as feeder, transporter, IME, inverter, finisher and need to arrive at the finisher in order. Thus, sheet $(n + 1)$ needs to be stacked in the same finisher with sheet n of the same job, but needs to arrive at the finisher right after sheet n (no other sheet stacked in between those two consecutive sheets). Since IMEs are heterogeneous (mixture of color and mono) and can run at different speeds, optimizing the operation of this printer for a mixture of print jobs, each of them is an arbitrary mixture of color/b&w pages that are either simplex (one-sided print) or duplex (two-sided print) is a hard problem.

Use [this already prepared session](#) to solve one problem instance. How many nodes did the planner expanded? How many did it generate but did not manage to expand? Post those stats in the forum!

End of tutorial worksheet